

Rezolvare detaliată - Subiect 02

Programare Avansată pe Obiecte în Java
Colocviu - Sesiunea iunie 2024, varianta 1

Rezumat răspunsuri

Exercițiu	Răspuns	Tema principală
I.1	c*	exceptii, try-with-resources, finally
I.2	d	clase imutabile, copiere defensivă
I.3	a	LinkedHashSet, equals, hashCode, String
I.4	c	upcasting, câmpuri ascunse, apeluri super
I.5	b	interfețe, Function, lambda, erori de compilare

Observație pentru I.1

La I.1, dacă fișierele marcate ca vid sunt cu adevărat goale, metoda `Scanner.nextInt()` aruncă `NoSuchElementException`, care nu este prinsă de cod. Atunci niciuna dintre variantele din grilă nu este exactă. În logica variantelor propuse de subiect, se consideră probabil că fișierul vid produce `InputMismatchException`; de aceea răspunsul de grilă este **c**.

1. Subiectul I - grilă, rezolvare explicată

1.1. Exercițiul I.1 - excepții, finally și propagare

Codul esențial este:

```

1 for (int i = 1; i <= 5; i++)
2     try (Scanner sc = new Scanner(new File("file" + i + ".txt"))) {
3         System.out.print(sc.nextInt());
4     } catch (FileNotFoundException e) {
5         System.out.print("!");
6         throw e;
7     } catch (InputMismatchException e) {
8         System.out.print("?");
9     } finally {
10        System.out.print("F");
11    }

```

Fișierele existente sunt:

Fișier	Conținut
file1.txt	vid
file2.txt	numărul 2
file3.txt	vid
file4.txt	numărul 4
file5.txt	nu există

În interpretarea variantelor de răspuns, fișierele vide duc la blocul catch (InputMismatchException) deci se afișează ?. Pentru fiecare iterație se execută apoi și blocul finally, deci se adaugă F.

i	Ce se întâmplă	Ce se afișează
1	fișier vid, eroare de citire număr	?F
2	se citește corect numărul 2	2F
3	fișier vid, eroare de citire număr	?F
4	se citește corect numărul 4	4F
5	fișierul nu există, se prinde FileNotFoundException, se re-aruncă	!F și programul se termină

Rezultatul din logica grilei este:

```

1 ?F2F?F4F!F

```

iar execuția se termină cu excepție, deoarece în blocul catch (FileNotFoundException e) se face:

```

1 throw e;

```

Răspuns corect: c) se va afișa ?F2F?F4F!F și executarea se va termina cu o excepție

Idee de examen: Blocul finally se execută indiferent dacă în try a apărut sau nu o excepție. Dacă în catch se face throw, excepția continuă să se propage după ce se execută finally.

Atenție tehnică

În Java real, pentru un fișier complet gol, `Scanner.nextInt()` aruncă `NoSuchElementException`, nu `InputMismatchException`. În acel caz s-ar afișa doar F și programul s-ar opri la `file1.txt`. Pentru examen, însă, trebuie urmărită logica variantelor date.

1.2. Exercițiul 1.2 - clase imutabile

Codul este:

```

1 class A {
2     Integer intA;
3     public A(Integer i) { intA = i; }
4     public void set(Integer i) { intA = i; }
5     public A(A other) { intA = other.intA; }
6 }
7
8 class B {
9     A object;
10    public B(A init) { object = new A(init); }
11    public B(B other) { object = other.object; }
12    public A getA() { return object; }
13 }
14
15 class C {
16     B object;
17    public C(B init) { object = init; }
18    public B getB() { return new B(object); }
19 }

```

Verificăm fiecare clasă.

Clasă	Explicație	Imutabilă?
A	Are metodă <code>set(Integer i)</code> , deci starea obiectului poate fi schimbată după construire.	Nu
B	În constructorul cu A face o copie, dar are <code>getA()</code> care returnează direct referința internă spre un obiect A mutabil. Deci cineva poate face <code>b.getA().set(10)</code> și modifică interiorul lui B. În plus, constructorul de copiere <code>B(B other)</code> copiază doar referința.	Nu
C	Constructorul primește un B și îl atribuie direct: <code>object = init</code> . Deci păstrează o referință externă spre un obiect mutabil. Metoda <code>getB()</code> întoarce <code>new B(object)</code> , dar constructorul <code>B(B other)</code> este superficial, deci nu protejează complet obiectul intern.	Nu

Prin urmare, niciuna dintre cele trei clase nu este imutabilă.

Răspuns corect: **d) 0**

Idee de examen: Pentru o clasă imutabilă: câmpurile trebuie să fie private și final, nu există setteri, iar pentru obiectele mutabile trebuie făcute copii defensive în constructor și în getteri.

1.3. Exercițiul 1.3 - LinkedHashSet, equals, hashCode

Codul important este:

```

1 class A {
2     String sir;
3     public A(String sir) { this.sir = sir; }
4     public boolean equals(Object obj) {
5         return sir.substring(1) == ((A)obj).sir.substring(1);
6     }
7     public int hashCode() { return (int)sir.charAt(0); }
8     public String toString() { return sir + " "; }
9 }

```

Se adaugă în LinkedHashSet următoarele obiecte:

```

1 lhs.add(new A("examen"));
2 lhs.add(new A("Java"));
3 lhs.add(new A("Examen"));
4 lhs.add(new A("PA0"));
5 lhs.add(new A("Java"));

```

Mai întâi calculăm hashCode():

Șir	Primul caracter	hashCode
examen	'e'	codul caracterului e
Java	'J'	codul caracterului J
Examen	'E'	codul caracterului E
PA0	'P'	codul caracterului P
Java	'J'	codul caracterului J

Doar cele două obiecte cu "Java" ajung sigur în același bucket, deoarece au același prim caracter. Pentru a decide dacă al doilea "Java" se mai adaugă, se apelează equals. Metoda equals este greșită:

```

1 return sir.substring(1) == ((A)obj).sir.substring(1);

```

Ea compară cu == două obiecte String, nu conținutul lor. substring(1) creează un șir nou, iar == compară referințele. Deci, chiar dacă textele sunt egale, referințele pot fi diferite, iar equals întoarce false. Astfel, al doilea "Java" nu este eliminat.

LinkedHashSet păstrează ordinea de inserare. Se afișează:

```

1 examen Java Examen PA0 Java

```

Răspuns corect: **a) examen Java Examen PA0 Java**

Idee de examen: Pentru șiruri, conținutul se compară cu equals, nu cu ==. LinkedHashSet elimină duplicatele ca un HashSet, dar păstrează ordinea de inserare.

1.4. Exercițiul 1.4 - upcasting, câmpuri ascunse și super

Codul este:

```

1 class A {
2     int x = 10;
3     public A(int x) { this.x = x; }
4     int f(int t) { return x + t; }
5 }
6
7 class B extends A {
8     int x = 30;
9     public B() { super(20); }
10    int f(int t) { return t + super.f(10*x); }

```

```

11 }
12
13 public class Test {
14     public static void main(String[] args) {
15         A ob = new B();
16         System.out.println(ob.f(ob.x));
17     }
18 }

```

Avem:

```
1 A ob = new B();
```

Deci:

- tip declarat: A;
- tip real: B.

Constructorul lui B apelează:

```
1 super(20);
```

Deci partea de A a obiectului are:

```
1 A.x = 20
```

Clasa B are propriul câmp:

```
1 B.x = 30
```

Apoi se execută:

```
1 ob.f(ob.x)
```

Pentru `ob.x`, câmpurile nu sunt polimorfe. Se folosește tipul declarat al referinței, adică A. Deci:

```
1 ob.x = 20
```

Pentru `ob.f(...)`, metodele de instanță sunt polimorfe. Se apelează metoda din tipul real, adică B.f:

```
1 B.f(20)
```

Calculăm B.f(20):

```
1 return t + super.f(10*x);
```

În metoda din B, numele simplu `x` se referă la câmpul lui B, deci `x = 30`. Prin urmare:

```
1 10*x = 300
```

Apoi:

```
1 super.f(300)
```

apelează metoda `f` din A. În A.f, câmpul `x` este câmpul lui A, adică 20:

```
1 A.f(300) = 20 + 300 = 320
```

Deci:

```
1 B.f(20) = 20 + 320 = 340
```

Răspuns corect: **c) 340**

Idee de examen: La upcasting, câmpurile se aleg după tipul declarat al referinței, iar metodele de instanță se aleg după tipul real al obiectului.

1.5. Exercițiul 1.5 - interfețe, Function și lambda

Codul este:

```

1 interface Functie { Function<Integer, Double> f = x -> 1.0/x; }
2
3 class A implements Functie{
4     public double f(int x) { return 2.0 / x; }
5 }
6
7 class Test {
8     static void afisare(Functie f, int t) {
9         System.out.println(f.f.apply(t));
10        System.out.println(f.f(t));
11    }
12
13    public static void main(String[] args) {
14        afisare(null, 10);
15        afisare(new A().f, 20);
16        afisare(null, (int)new A().f(1));
17        afisare(new Functie() {}, 10);
18        afisare(x -> 3.0 / x, 20);
19    }
20 }
```

Interfața Functie nu are metode abstracte. Ea conține doar un câmp:

```
1 Function<Integer, Double> f = x -> 1.0/x;
```

În interfețe, câmpurile sunt implicit:

```
1 public static final
```

Deci `Functie.f` este o constantă de tip `Function<Integer, Double>`. Interfața `Functie` nu este interfață funcțională, fiindcă nu are nicio metodă abstractă.

Analizăm erorile din clasa `Test`.

Instrucțiune	Explicație	Eroare?
<code>f.f.apply(t)</code>	Se accesează câmpul static <code>f</code> din interfață și se apelează <code>apply</code> . Este permis, chiar dacă stilistic ar fi mai corect <code>Functie.f.apply(t)</code> .	Nu
<code>f.f(t)</code>	Variabila <code>f</code> are tipul <code>Functie</code> . Interfața <code>Functie</code> nu declară metoda <code>f(int)</code> . Câmpul <code>f</code> nu se apelează ca metodă directă.	Da
<code>afisare(null, 10)</code>	<code>null</code> poate fi transmis pentru o referință de tip <code>Functie</code> .	Nu
<code>afisare(new A().f, 20)</code>	<code>new A().f</code> accesează câmpul <code>f</code> de tip <code>Function<Integer, Double></code> . Dar metoda <code>afisare</code> cere un argument de tip <code>Functie</code> .	Da

<code>afisare(null, (int)new A().f(1))</code>	Aici se apelează metoda <code>f(int)</code> din clasa <code>A</code> ; rezultatul <code>double</code> este convertit explicit la <code>int</code> .	Nu
<code>afisare(new Functie() {}, 10)</code>	Se poate crea o clasă anonimă pentru o interfață fără metode abstracte. Nu trebuie implementat nimic.	Nu
<code>afisare(x -> 3.0 / x, 20)</code>	Lambda expresia poate fi folosită doar când tipul țintă este o interfață funcțională. <code>Functie</code> nu este interfață funcțională.	Da

Deci sunt 3 erori de compilare în clasa `Test`.

Răspuns corect: b) 3

Idee de examen: O lambda expresie are nevoie de o interfață funcțională, adică o interfață cu exact o metodă abstractă. Câmpurile din interfețe nu sunt metode abstracte, ci constante `public static final`.

2. Subiectul II - clasa Imobil și stream-uri

Cerința: definim complet clasa Imobil, creăm o listă cu cel puțin 3 obiecte și rezolvăm cerințele folosind stream-uri și lambda expresii.

2.1. Clasa Imobil

```
1 import java.util.Objects;
2
3 public class Imobil {
4     private String tip;
5     private String localitate;
6     private int nrCamere;
7     private double suprafata;
8     private double pret;
9
10    public Imobil() {
11    }
12
13    public Imobil(String tip, String localitate, int nrCamere,
14                  double suprafata, double pret) {
15        this.tip = tip;
16        this.localitate = localitate;
17        this.nrCamere = nrCamere;
18        this.suprafata = suprafata;
19        this.pret = pret;
20    }
21
22    public String getTip() {
23        return tip;
24    }
25
26    public void setTip(String tip) {
27        this.tip = tip;
28    }
29
30    public String getLocalitate() {
31        return localitate;
32    }
33
34    public void setLocalitate(String localitate) {
35        this.localitate = localitate;
36    }
37
38    public int getNrCamere() {
39        return nrCamere;
40    }
41
42    public void setNrCamere(int nrCamere) {
43        this.nrCamere = nrCamere;
44    }
45
46    public double getSuprafata() {
47        return suprafata;
48    }
49
50    public void setSuprafata(double suprafata) {
51        this.suprafata = suprafata;
52    }
53 }
```



```

54 public double getPret() {
55     return pret;
56 }
57
58 public void setPret(double pret) {
59     this.pret = pret;
60 }
61
62 @Override
63 public String toString() {
64     return "Imobil{" +
65         "tip='" + tip + '\'' +
66         ", localitate='" + localitate + '\'' +
67         ", nrCamere=" + nrCamere +
68         ", suprafata=" + suprafata +
69         ", pret=" + pret +
70         '}';
71 }
72
73 @Override
74 public boolean equals(Object o) {
75     if (this == o) return true;
76     if (!(o instanceof Imobil imobil)) return false;
77     return nrCamere == imobil.nrCamere &&
78         Double.compare(suprafata, imobil.suprafata) == 0 &&
79         Double.compare(pret, imobil.pret) == 0 &&
80         Objects.equals(tip, imobil.tip) &&
81         Objects.equals(localitate, imobil.localitate);
82 }
83
84 @Override
85 public int hashCode() {
86     return Objects.hash(tip, localitate, nrCamere, suprafata, pret);
87 }
88 }

```

2.2. Rezolvarea cerințelor cu stream-uri

```

1 import java.util.*;
2 import java.util.stream.Collectors;
3
4 public class TestImobil {
5     public static void main(String[] args) {
6         List<Imobil> imobile = new ArrayList<>();
7         imobile.add(new Imobil("apartament", "Bucuresti", 3, 72.5, 450000));
8         imobile.add(new Imobil("garsoniera", "Bucuresti", 1, 35.0, 250000));
9         imobile.add(new Imobil("casa", "Brasov", 4, 120.0, 620000));
10        imobile.add(new Imobil("apartament", "Cluj", 3, 68.0, 490000));
11        imobile.add(new Imobil("apartament", "Bucuresti", 4, 95.0, 500000));
12
13        // a) Imobile cu cel puțin 3 camere si pret <= 500000,
14        // sortate crescator dupa suprafata.
15        System.out.println("a) Imobile filtrate si sortate dupa suprafata:");
16        imobile.stream()
17            .filter(i -> i.getNrCamere() >= 3)
18            .filter(i -> i.getPret() <= 500000)
19            .sorted(Comparator.comparingDouble(Imobil::getSuprafata))
20            .forEach(System.out::println);
21
22        // b) Localitati distincte.

```

```
23     System.out.println("\nb) Localitati distincte:");
24     imobile.stream()
25         .map(Imobil::getLocalitate)
26         .distinct()
27         .forEach(System.out::println);
28
29     // c) Lista cu imobile din Bucuresti, cu pret intre 300000 si 500000.
30     System.out.println("\nc) Lista imobile Bucuresti, pret intre 300000 si
31     500000:");
32     List<Imobil> bucuresti = imobile.stream()
33         .filter(i -> i.getLocalitate().equalsIgnoreCase("Bucuresti"))
34         .filter(i -> i.getPret() >= 300000 && i.getPret() <= 500000)
35         .collect(Collectors.toList());
36
37     bucuresti.forEach(System.out::println);
38
39     // d) Pentru fiecare localitate distincta, imobilele din acea localitate.
40     System.out.println("\nd) Imobile grupate pe localitati:");
41     Map<String, List<Imobil>> imobilePeLocalitati = imobile.stream()
42         .collect(Collectors.groupingBy(Imobil::getLocalitate));
43
44     imobilePeLocalitati.forEach((localitate, lista) -> {
45         System.out.println("Localitate: " + localitate);
46         lista.forEach(System.out::println);
47     });
48 }
```

2.3. Explicații rapide pentru stream-uri

- `filter` păstrează doar obiectele care respectă condiția.
- `sorted(Comparator.comparingDouble(...))` sortează crescător după un câmp numeric de tip `double`.
- `map(Imobil::getLocalitate)` transformă stream-ul de imobile într-un stream de localități.
- `distinct()` elimină duplicatele.
- `collect(Collectors.toList())` salvează rezultatul într-o listă.
- `groupingBy(Imobil::getLocalitate)` creează un `Map<String, List<Imobil>>`, adică pentru fiecare localitate lista imobilelor aferente.

3. Subiectul III - fișiere text și fire de executare

Cerința: fiecare linie din fișier conține:

```
1 tip,localitate,nrCamere,suprafata,pret
```

Trebuie să calculăm pentru un fișier numărul imobilelor care au:

```
1 suprafata > smin
2 nrCamere >= cmin
```

Folosim câte un fir de executare pentru fiecare fișier: AgentieRS_1.txt și AgentieRS_2.txt.

3.1. Clasă pentru numărarea imobilelor dintr-un fișier

```
1 import java.io.BufferedReader;
2 import java.io.IOException;
3 import java.nio.file.Files;
4 import java.nio.file.Path;
5
6 class NumaratorImobile extends Thread {
7     private final String numeFisier;
8     private final double smin;
9     private final int cmin;
10    private int rezultat;
11
12    public NumaratorImobile(String numeFisier, double smin, int cmin) {
13        this.numeFisier = numeFisier;
14        this.smin = smin;
15        this.cmin = cmin;
16    }
17
18    @Override
19    public void run() {
20        rezultat = 0;
21
22        try (BufferedReader br = Files.newBufferedReader(Path.of(numeFisier))) {
23            String linie;
24
25            while ((linie = br.readLine()) != null) {
26                String[] v = linie.split(",");
27
28                // v[0] = tip
29                // v[1] = localitate
30                // v[2] = nrCamere
31                // v[3] = suprafata
32                // v[4] = pret
33                int nrCamere = Integer.parseInt(v[2].trim());
34                double suprafata = Double.parseDouble(v[3].trim());
35
36                if (suprafata > smin && nrCamere >= cmin) {
37                    rezultat++;
38                }
39            }
40        } catch (IOException e) {
41            throw new RuntimeException(e);
42        }
43    }
44
45    public int getRezultat() {
46        return rezultat;
47    }
48 }
```

```
47 }  
48 }
```

3.2. Programul principal

```
1 import java.util.Scanner;  
2  
3 public class TestRentSale {  
4     public static void main(String[] args) throws InterruptedException {  
5         Scanner scanner = new Scanner(System.in);  
6  
7         System.out.print("smin = ");  
8         double smin = scanner.nextDouble();  
9  
10        System.out.print("cmin = ");  
11        int cmin = scanner.nextInt();  
12  
13        NumaratorImobile fir1 = new NumaratorImobile("AgentieRS_1.txt", smin, cmin)  
14        ;  
15        NumaratorImobile fir2 = new NumaratorImobile("AgentieRS_2.txt", smin, cmin)  
16        ;  
17  
18        fir1.start();  
19        fir2.start();  
20  
21        fir1.join();  
22        fir2.join();  
23  
24        int total = fir1.getRezultat() + fir2.getRezultat();  
25        System.out.println("Numar total imobile: " + total);  
26    }  
27 }
```

3.3. De ce folosim join()?

start() pornește firele, dar nu așteaptă terminarea lor. Dacă am calcula totalul imediat după start(), este posibil ca firele să nu fi terminat citirea fișierelor. De aceea folosim:

```
1 fir1.join();  
2 fir2.join();
```

Aceste apeluri obligă firul principal să aștepte terminarea celor două fire, apoi rezultatele sunt corecte.

Idee de examen: În exercițiile cu thread-uri: start() pornește firul, run() conține codul executat pe fir, iar join() așteaptă terminarea firului.

4. Subiectul IV - clasă singleton pentru citire CSV

Cerința: avem clasa Persoana, care memorează:

- nume - String;
- varsta - număr natural;
- salariuMediuAnual - număr real.

Trebuie definită o clasă singleton CitireInformatiiPersoane care citește dintr-un fișier CSV și întoarce un ArrayList<Persoana>.

4.1. Clasa Persoana - variantă completă

Deși enunțul spune că este definită complet, la examen putem scrie o variantă minimală ca să se înțeleagă programul.

```
1 public class Persoana {
2     private String nume;
3     private int varsta;
4     private double salariuMediuAnual;
5
6     public Persoana(String nume, int varsta, double salariuMediuAnual) {
7         this.nume = nume;
8         this.varsta = varsta;
9         this.salariuMediuAnual = salariuMediuAnual;
10    }
11
12    public String getNume() {
13        return nume;
14    }
15
16    public void setNume(String nume) {
17        this.nume = nume;
18    }
19
20    public int getVarsta() {
21        return varsta;
22    }
23
24    public void setVarsta(int varsta) {
25        this.varsta = varsta;
26    }
27
28    public double getSalariuMediuAnual() {
29        return salariuMediuAnual;
30    }
31
32    public void setSalariuMediuAnual(double salariuMediuAnual) {
33        this.salariuMediuAnual = salariuMediuAnual;
34    }
35
36    @Override
37    public String toString() {
38        return "Persoana{" +
39            "nume='" + nume + '\'' +
40            ", varsta=" + varsta +
41            ", salariuMediuAnual=" + salariuMediuAnual +
42            '}';
43    }
44 }
```

4.2. Clasa singleton CitireInformatiiPersoane

Formatul unei linii CSV este:

```
1 nume,varsta,salariuMediuAnual
```

Implementare:

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4 import java.util.ArrayList;
5
6 public class CitireInformatiiPersoane {
7     private static final CitireInformatiiPersoane INSTANCE =
8         new CitireInformatiiPersoane();
9
10    private CitireInformatiiPersoane() {
11    }
12
13    public static CitireInformatiiPersoane getInstance() {
14        return INSTANCE;
15    }
16
17    public ArrayList<Persoana> citestePersoane(String numeFisier)
18        throws IOException {
19        ArrayList<Persoana> persoane = new ArrayList<>();
20
21        try (BufferedReader br = new BufferedReader(new FileReader(numeFisier))) {
22            String linie;
23
24            while ((linie = br.readLine()) != null) {
25                String[] valori = linie.split(",");
26
27                String nume = valori[0].trim();
28                int varsta = Integer.parseInt(valori[1].trim());
29                double salariu = Double.parseDouble(valori[2].trim());
30
31                persoane.add(new Persoana(nume, varsta, salariu));
32            }
33        }
34
35        return persoane;
36    }
37 }
```

4.3. Exemplu de utilizare

```
1 import java.io.IOException;
2 import java.util.ArrayList;
3
4 public class TestCitirePersoane {
5     public static void main(String[] args) throws IOException {
6         CitireInformatiiPersoane cititor =
7             CitireInformatiiPersoane.getInstance();
8
9         ArrayList<Persoana> persoane =
10             cititor.citestePersoane("persoane.csv");
11
12         persoane.forEach(System.out::println);
13     }
14 }
```

14 }

4.4. De ce este singleton?

Clasa este singleton pentru că:

- constructorul este `private`, deci nu putem face `new CitireInformatiiPersoane()` din exterior;
- există o singură instanță statică:

```
1 private static final CitireInformatiiPersoane INSTANCE =  
2     new CitireInformatiiPersoane();
```

- accesul se face prin metoda statică:

```
1 public static CitireInformatiiPersoane getInstance()
```

Idee de examen: La singleton: constructor `private`, instanță statică unică, metodă publică statică `getInstance()`.